Task 1: Create a tomcat container on 8080 and deploy sample application in tomcat.

Objective

To create a Tomcat Docker container, expose it on port 8080, and deploy a sample HTML application.

Steps Used

Step 1: Verify Docker installation

```
sudo docker ps
```

Checked the running Docker containers and identified that port 8080 was already occupied by the `jenkins-lab` container.

Step 2: Stop and remove the existing Jenkins container

```
sudo docker stop 43d3df8cb35e
sudo docker rm 43d3df8cb35e
```

Freed port 8080 for the Tomcat container. Note: this stopped and removed the Jenkins container.

Step 3: Create the application directory

```
mkdir -p ~/docker-02/webapps
cd ~/docker-02/webapps
```

Created a directory to store the sample web application.

Step 4: Create the sample HTML file

```
nano index.html
```

Added the following HTML content:

```
<!DOCTYPE html>
<html>
<head>
    <title>Docker</title>
</head>
<body>
```

```
    <h1>Welcome to Docker!</h1>
    <p>Sample application deployed on Tomcat by Asifshaik.</p>
</body>
</html>
```

Saved the file as `index.html`.

Step 5: Download the Tomcat image

```
sudo docker pull tomcat:10.1
```

Downloaded the Tomcat 10.1 Docker image.

Step 6: Remove the failed Tomcat container

```
sudo docker rm tomcat-8080
```

Removed the previously created container that failed because port 8080 was already allocated.

Step 7: Create and start the Tomcat container

```
sudo docker run -d \
  --name tomcat-8080 \
  -p 8080:8080 \
  -v "$HOME/docker-02/webapps:/usr/local/tomcat/webapps/ROOT" \
  tomcat:10.1
```

- `-d`: Runs the container in the background.
- `--name`: Assigns the name `tomcat-8080`.
- `-p 8080:8080`: Maps host port 8080 to container port 8080.
- `-v`: Mounts the sample application directory into Tomcat's ROOT application directory.
- `tomcat:10.1`: Specifies the Tomcat image and version.

Step 8: Verify the running container

```
sudo docker ps
```

Confirmed that the `tomcat-8080` container was running and port 8080 was mapped correctly.

Step 9: Test the application

```
curl http://localhost:8080/
```

Verified that the HTML content was returned successfully.

```
ubuntu@ip-172-31-29-148:~$ sudo docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED        STATUS        PORTS                                                                                  NAMES
43d3df8cb35e   jenkins/jenkins:lts-jdk21      "/usr/bin/tini -- /u…"   22 hours ago   Up 8 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 50000/tcp                                 jenkins-lab
ubuntu@ip-172-31-29-148:~$
ubuntu@ip-172-31-29-148:~$ sudo docker pull tomcat:10.1
10.1: Pulling from library/tomcat
4f4fb700ef54: Pull complete
4e07a0f12b2c: Pull complete
06ad70e463aa: Pull complete
5dec61325458: Pull complete
f39241fdc590: Pull complete
41da82ce3852: Pull complete
130739ca07b1: Pull complete
1b1b01e8b066: Pull complete
e3c722fd9f88: Download complete
ca23e54f4eb1: Download complete
Digest: sha256:574ed18cff4d597067f98b80426fc64dbfb217f1dbc5b0992b9383f008bf8874
Status: Downloaded newer image for tomcat:10.1
docker.io/library/tomcat:10.1
ubuntu@ip-172-31-29-148:~$ mkdir -p ~/docker-02/webapps
ubuntu@ip-172-31-29-148:~$ cd ~/docker-02/webapps
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ vi index.html
```

```
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED        STATUS          PORTS                                                                                  NAMES
43d3df8cb35e   jenkins/jenkins:lts-jdk21      "/usr/bin/tini -- /u…"   22 hours ago   Up 14 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 50000/tcp                                 jenkins-lab
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker stop 43d3df8cb35e
43d3df8cb35e
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker ps -a
CONTAINER ID   IMAGE                          COMMAND                  CREATED         STATUS                       PORTS     NAMES
f85ec063bf20   tomcat:10.1                    "catalina.sh run"        2 minutes ago   Created                              tomcat-8080
43d3df8cb35e   jenkins/jenkins:lts-jdk21      "/usr/bin/tini -- /u…"   22 hours ago    Exited (143) 15 seconds ago          jenkins-lab
91a5dfdcb3fb   nginx:latest                   "/docker-entrypoint.…"   22 hours ago    Exited (137) 17 hours ago            nginx-lab
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ docker rm 43d3df8cb35e
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker rm 43d3df8cb35e
43d3df8cb35e
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ docker ps -a
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS                       PORTS     NAMES
f85ec063bf20   tomcat:10.1    "catalina.sh run"        3 minutes ago   Created                              tomcat-8080
91a5dfdcb3fb   nginx:latest   "/docker-entrypoint.…"   22 hours ago    Exited (137) 17 hours ago            nginx-lab
ubuntu@ip-172-31-29-148:~/docker-02/webapps$
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker rm tomcat-8080
tomcat-8080
ubuntu@ip-172-31-29-148:~/docker-02/webapps$
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker run -d \
  --name tomcat-8080 \
  -p 8080:8080 \
  -v "$HOME/docker-02/webapps:/usr/local/tomcat/webapps/ROOT" \
  tomcat:10.1
f5abf2f8aeb8f58a05f3bd923a13a309b1b059dd87241ce81a2748a1f3e25428
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker ps
CONTAINER ID   IMAGE         COMMAND             CREATED         STATUS         PORTS                                                                  NAMES
f5abf2f8aeb8   tomcat:10.1   "catalina.sh run"   9 seconds ago   Up 9 seconds   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp                            tomcat-8080
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ curl http://localhost:8080/
<!DOCTYPE html>
<html>
<head>
    <title>Docker</title>
</head>
<body>
    <h1>Welcome to Docker!</h1>
    <p>Sample application deployed on Tomcat by Asifshaik.</p>
</body>
</html>
ubuntu@ip-172-31-29-148:~/docker-02/webapps$
```



Welcome to Docker!

Sample application deployed on Tomcat by Asifshaik.

Observation

- The Tomcat 10.1 container started successfully.
- Port 8080 was mapped from the EC2 host to the container.
- The sample HTML file was mounted into Tomcat's ROOT directory.
- The `curl` command returned the expected HTML content, confirming that the application was accessible locally.

Conclusion

Successfully created and deployed a Tomcat Docker container on port 8080 and hosted a sample HTML application. Verified the deployment using Docker container status and the `curl` command. This task demonstrated Docker image management, container creation, port mapping, bind mounts, and basic application deployment.

Task 2: Create a Docker volume and deploy Tomcat on port 8081.

Objective

To create a Docker volume, mount it to a Tomcat container, and deploy a sample HTML application using persistent storage.

Steps Used

Step 1: Create a Docker volume

```
sudo docker volume create tomcat-data
```

Created a named Docker volume called `tomcat-data` to store application files independently of the container.

Step 2: Verify the Docker volume

```
sudo docker volume ls
sudo docker volume inspect tomcat-data
```

Verified that the volume was created successfully and inspected its details, including its name, driver, and mount point.

Step 3: Run a Tomcat container on port 8081

```
sudo docker run -d \
  --name tomcat-8081 \
  -p 8081:8080 \
  -v tomcat-data:/usr/local/tomcat/webapps \
  tomcat:10.1
```

Created and started the `tomcat-8081` container.

- `-d`: Runs the container in the background.
- `--name`: Assigns the container name.
- `-p 8081:8080`: Maps host port 8081 to container port 8080.
- `-v`: Mounts the named volume into Tomcat's application directory.

Step 4: Download the Alpine image

```
sudo docker pull alpine:latest
```

Downloaded the lightweight Alpine Linux image to use a temporary container for creating the sample application.

Step 5: Mount the volume in a temporary container

```
sudo docker run --rm -it \
  -v tomcat-data:/data \
  alpine:latest sh
```

Started a temporary Alpine container and mounted `tomcat-data` at `/data`.

Step 6: Create the application directory

Inside the Alpine shell, executed:

```
mkdir -p /data/ROOT
```

Created the `ROOT` directory inside the mounted volume for Tomcat's root web application.

Step 7: Create the sample HTML file

```
echo '<html><body><h1>Welcome to Docker - Tomcat Volume by
Asifshaik!</h1></body></html>' > /data/ROOT/index.html
```

Created `index.html` inside the volume.

Step 8: Verify the HTML file

```
cat /data/ROOT/index.html
```

Confirmed that the HTML content was written correctly.

Step 9: Exit the temporary container

```
exit
```

Exited the Alpine shell. Docker automatically removed the temporary container because the `--rm` option was used. The named volume and its data remained intact.

Step 10: Test the Tomcat application

`curl` http://localhost:8081/

Verified that Tomcat returned the expected HTML content.

```
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker volume create tomcat-data
tomcat-data
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker volume ls
DRIVER    VOLUME NAME
local     jenkins_home
local     tomcat-data
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker run -d \
  --name tomcat-8081 \
  -p 8081:8080 \
  -v tomcat-data:/usr/local/tomcat/webapps \
  tomcat:10.1
21009c52af37d745b5f2ec93e40376a03e4bebb6b0435a88f7572d4ef73f574f
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker ps
CONTAINER ID   IMAGE         COMMAND             CREATED          STATUS          PORTS                                       NAMES
21009c52af37   tomcat:10.1   "catalina.sh run"   13 seconds ago   Up 13 seconds   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   tomcat-8081
f5abf2f8aeb8   tomcat:10.1   "catalina.sh run"   15 minutes ago   Up 15 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   tomcat-8080
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker pull alpine:latest
latest: Pulling from library/alpine
e2de96513ba9: Pull complete
6d0606d1815c: Download complete
797dd00a0fc7: Download complete
Digest: sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker volume inspect tomcat-data
[
    {
        "CreatedAt": "2026-10-13T10:46:18Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/tomcat-data/_data",
        "Name": "tomcat-data",
        "Options": null,
        "Scope": "local"
    }
]
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ sudo docker run --rm -it \
  -v tomcat-data:/data \
  alpine:latest sh
/ # mkdir -p /data/ROOT
/ # echo '<html><body><h1>Welcome to Docker - Tomcat Volume by Asifshaik!</h1></body></html>' > /data/ROOT/index.html
/ # cat /data/ROOT/index.html
<html><body><h1>Welcome to Docker - Tomcat Volume by Asifshaik!</h1></body></html>
/ # exit
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ curl http://localhost:8081/
<html><body><h1>Welcome to Docker - Tomcat Volume by Asifshaik!</h1></body></html>
ubuntu@ip-172-31-29-148:~/docker-02/webapps$
```



Welcome to Docker - Tomcat Volume by Asifshaik!

Observation

- The `tomcat-data` volume was created successfully.
- The `tomcat-8081` container started with host port 8081 mapped to container port 8080.
- The sample HTML file was stored inside the named volume.
- The `curl` command returned the expected HTML content, confirming successful application deployment.
- The application data remained available after the temporary Alpine container was removed.

Conclusion

Successfully created a Docker volume and mounted it into a Tomcat container running on port 8081. Deployed and tested a sample HTML application using the named volume, demonstrating persistent storage, volume mounting, port mapping, and containerized web application deployment.

Task 3: Limit Nginx Container Memory to 500 MB

Objective

To run an Nginx Docker container with a maximum memory limit of 500 MiB and verify the limit using Docker commands.

Steps Used

Step 1: Run the Nginx Container with a Memory Limit

Executed the following command to start an Nginx container with a 500 MiB memory limit and expose it on port 8082.

```
sudo docker run -d --name nginx-lab --memory=500m -p 8082:80 nginx:latest
```

Explanation:

- `docker run` — Creates and starts a container.
- `-d` — Runs the container in the background.
- `--name nginx-lab` — Assigns the container name `nginx-lab`.
- `--memory=500m` — Sets the maximum memory limit to 500 MiB.
- `-p 8082:80` — Maps host port 8082 to container port 80.
- `nginx:latest` — Uses the latest Nginx image.

Step 2: Verify the Container

```
sudo docker ps
```

Confirmed that the `nginx-lab` container was running.

Step 3: Verify the Memory Limit

```
sudo docker inspect nginx-lab --format '{{.HostConfig.Memory}}'
```

Output:

```
524288000
```

This value represents 500 MiB in bytes.

Step 4: Monitor Memory Usage

```
sudo docker stats --no-stream nginx-lab
```

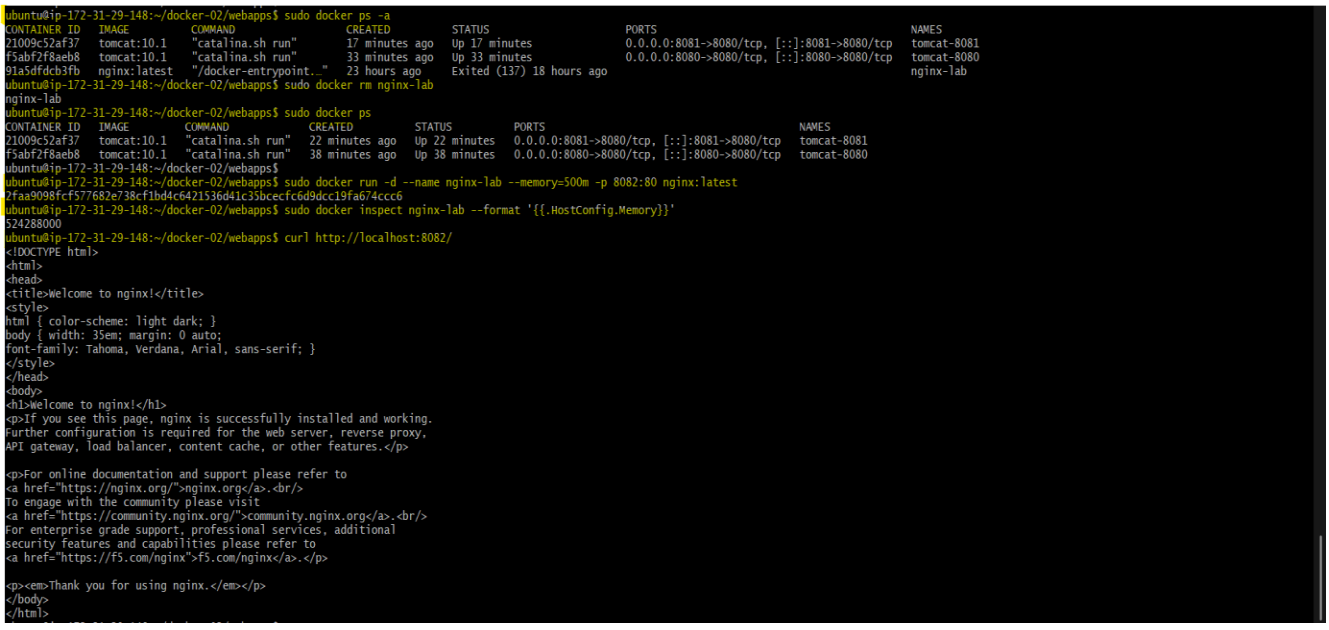
Observed output:

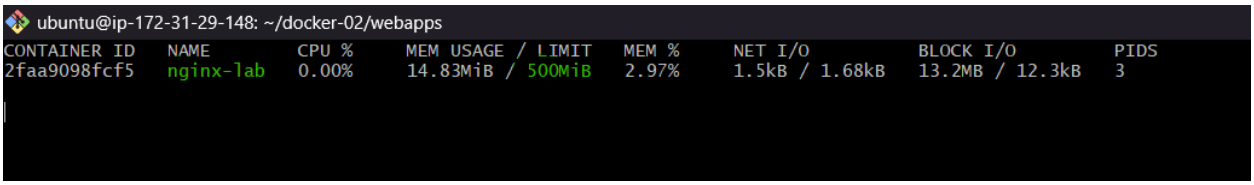
```
CONTAINER ID   NAME        CPU %   MEM USAGE / LIMIT   MEM %   NET I/O          BLOCK I/O       PIDS
2faa9098fcf5   nginx-lab   0.00%   14.83MiB / 500MiB   2.97%   1.5kB / 1.68kB   13.2MB / 12.3kB   3
```

Step 5: Verify the Nginx Web Page

```
curl http://localhost:8082/
```

The default Nginx welcome page HTML was returned, confirming that Nginx was responding on port 8082.





Observation

- The Nginx container started successfully.
- The memory limit was confirmed as `524288000` bytes, equivalent to 500 MiB.
- Docker Stats displayed current memory usage of `14.83MiB / 500MiB`, or 2.97% of the limit.
- The Nginx welcome page was accessible through port 8082.

Conclusion

Successfully created and verified an Nginx Docker container with a **500 MiB memory limit** using the `--memory=500m` option. The `docker inspect` and `docker stats` commands confirmed the configured limit and current memory usage.

Task 4: Create a sample docker file using below instructions.

1. **Base module as amazonlinux:latest**
2. **Maintainer you name**
3. **Install nginx**
4. **COPY one index.html file to image**
5. **Expose on port 80**
6. **Command to start the nginx container**

Objective

To create a custom Docker image using Amazon Linux, install Nginx, copy a custom HTML page, expose port 80, and run the container.

Steps Used

Step 1: Create the project directory

```
mkdir -p ~/docker-02/custom-nginx
```

```
cd ~/docker-02/custom-nginx
```

Step 2: Create the HTML file

Created `index.html` containing a custom welcome page.

Step 3: Create the Dockerfile

Created a file named `Dockerfile` with the following instructions:

```
FROM amazonlinux:latest
LABEL maintainer="Asifshaik"
RUN dnf install -y nginx && dnf clean all
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Explanation:

- `FROM`: Selects Amazon Linux as the base image.
- `LABEL`: Adds maintainer information.
- `RUN`: Installs Nginx and cleans package metadata.
- `COPY`: Copies the custom HTML page into Nginx's web directory.
- `EXPOSE`: Documents container port 80.
- `CMD`: Starts Nginx in the foreground.

Step 4: Build the custom image

```
sudo docker build -t asif-custom-nginx:1.0 .
```

The image was created successfully.

Image details:

- Image name: `asif-custom-nginx`

- Tag: `1.0`
- Image ID: `71660cc903e7`

Step 5: Run the container

```
sudo docker run -d \
  --name custom-nginx \
  -p 8083:80 \
  asif-custom-nginx:1.0
```

Step 6: Verify the container

```
sudo docker ps
```

The `custom-nginx` container was running with port mapping `8083:80`.

Step 7: Test the application

`curl` http://localhost:8083/

The command returned the custom HTML page containing the message **"Welcome to Custom Nginx!"**

```
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ mkdir -p ~/docker-02/custom-nginx
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ nano index.html
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ cat index.html
<!DOCTYPE html>
<html>
<head>
    <title>Custom Nginx</title>
</head>
<body>
    <h1>Welcome to Custom Nginx!</h1>
    <p>Custom Nginx image built using Amazon Linux by Asifshaik.</p>
</body>
</html>
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ cp ~/docker-02/webapps/index.html ~/docker-02/custom-nginx/index.html
ubuntu@ip-172-31-29-148:~/docker-02/webapps$ cd ~/docker-02/custom-nginx
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ pwd
/home/ubuntu/docker-02/custom-nginx
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ cat index.html
<!DOCTYPE html>
<html>
<head>
    <title>Custom Nginx</title>
</head>
<body>
    <h1>Welcome to Custom Nginx!</h1>
    <p>Custom Nginx image built using Amazon Linux by Asifshaik.</p>
</body>
</html>
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ nano Dockerfile
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ cat Dockerfile
FROM amazonlinux:latest

LABEL maintainer="Asifshaik"

RUN dnf install -y nginx && dnf clean all

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

```
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ sudo docker build -t asif-custom-nginx:1.0 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/6 : FROM amazonlinux:latest
latest: Pulling from library/amazonlinux
7234d0a9b6ba: Pulling fs layer
e55ddf391d4b: Download complete
38ee9afa4295: Download complete
7234d0a9b6ba: Download complete
7234d0a9b6ba: Pull complete
Digest: sha256:8ed3c0a996841537f75607e7d1de2114d8150391f75792e8da9268738547e73f
Status: Downloaded newer image for amazonlinux:latest
 ---> 8ed3c0a99684
Step 2/6 : LABEL maintainer="Asifshaik"
 ---> Running in 4a5b8166b1fe
 ---> Removed intermediate container 4a5b8166b1fe
 ---> ca6c876f77d0
Step 3/6 : RUN dnf install -y nginx && dnf clean all
 ---> Running in 4dfe7a3805fd
Amazon Linux 2023 repository                    108 MB/s |  81 MB     00:00
Last metadata expiration check: 0:00:12 ago on Tue Oct 13 11:28:25 2026.
Dependencies resolved.
================================================================================
 Package                  Arch    Version                   Repository    Size
================================================================================
Installing:
 nginx                    x86_64  1:1.30.5-1.amzn2023.0.1   amazonlinux   34 k
Installing dependencies:
 cracklib                 x86_64  2.9.6-27.amzn2023.0.2     amazonlinux   82 k
 dbus                     x86_64  1:1.12.28-1.amzn2023.0.1  amazonlinux  8.5 k
 dbus-broker              x86_64  32-1.amzn2023.0.2         amazonlinux  174 k
 dbus-common              noarch  1:1.12.28-1.amzn2023.0.1  amazonlinux   15 k
 device-mapper            x86_64  1.02.185-1.amzn2023.0.5   amazonlinux  137 k
 device-mapper-libs       x86_64  1.02.185-1.amzn2023.0.5   amazonlinux  177 k
 diffutils                x86_64  3.8-1.amzn2023.0.2        amazonlinux  370 k
 generic-logos-httpd      noarch  18.0.0-12.amzn2023.0.3    amazonlinux   19 k
 gperftools-libs          x86_64  2.9.1-1.amzn2023.0.3      amazonlinux  308 k
 gzip                     x86_64  1.12-1.amzn2023.0.1       amazonlinux  160 k
 kmod-libs                x86_64  29-2.amzn2023.0.6         amazonlinux   62 k
 libargon2                x86_64  20171227-9.amzn2023.0.2   amazonlinux   29 k
 libdb                    x86_64  5.3.28-49.amzn2023.0.2    amazonlinux  756 k
 libeconf                 x86_64  0.7.9-1.amzn2023.0.1      amazonlinux   36 k
 libfdisk                 x86_64  2.37.4-1.amzn2023.0.6     amazonlinux  152 k
 libpwquality             x86_64  1.4.4-6.amzn2023.0.2      amazonlinux  106 k
 libseccomp               x86_64  2.5.3-1.amzn2023.0.2      amazonlinux   71 k
 libselinux-utils         x86_64  3.4-5.amzn2023.0.2        amazonlinux  142 k
 libsemanage              x86_64  3.4-5.amzn2023.0.2        amazonlinux  121 k
 libunwind                x86_64  1.4.0-5.amzn2023.0.3      amazonlinux   66 k
 libutempter              x86_64  1.2.1-4.amzn2023.0.2      amazonlinux   26 k
 nginx-core               x86_64  1:1.30.5-1.amzn2023.0.1   amazonlinux  711 k
 nginx-filesystem         noarch  1:1.30.5-1.amzn2023.0.1   amazonlinux   10 k
 nginx-mimetypes          noarch  2.1.49-3.amzn2023.0.3     amazonlinux   21 k
 pam                      x86_64  1.5.1-8.amzn2023.0.8      amazonlinux  550 k
 policycoreutils          x86_64  3.4-6.amzn2023.0.3        amazonlinux  210 k
```

```
Complete!
9 files removed
 ---> Removed intermediate container 4dfe7a3805fd
 ---> 341c69bb65e9
Step 4/6 : COPY index.html /usr/share/nginx/html/index.html
 ---> 9818e29c3409
Step 5/6 : EXPOSE 80
 ---> Running in 227606e57c57
 ---> Removed intermediate container 227606e57c57
 ---> 3abfdae50aa9
Step 6/6 : CMD ["nginx", "-g", "daemon off;"]
 ---> Running in dc12804e5c59
 ---> Removed intermediate container dc12804e5c59
 ---> 71660cc903e7
Successfully built 71660cc903e7
Successfully tagged asif-custom-nginx:1.0
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ sudo docker images
                                                
```



Observation

- The custom Docker image built successfully.
- The Nginx container started successfully.
- Port 8083 on the EC2 host was mapped to port 80 inside the container.
- The custom HTML page was served correctly.

Conclusion

Successfully built and deployed a custom Nginx Docker image using Amazon Linux and a Dockerfile. Verified the running container and confirmed that the custom web page was accessible through port 8083.

Task 5: Push a Custom Docker Image to Docker Hub

Objective

To upload the custom Nginx Docker image to Docker Hub so it can be stored and pulled from other machines.

Steps Used

Step 1: Log in to Docker Hub

```
sudo docker login
```

Completed the browser-based authentication process.

Step 2: Tag the image

```
sudo docker tag asif-custom-nginx:1.0 asifshaik6558/docker-02-nginx:1.0
```

This assigned the Docker Hub repository name and tag to the existing local image.

Step 3: Push the image

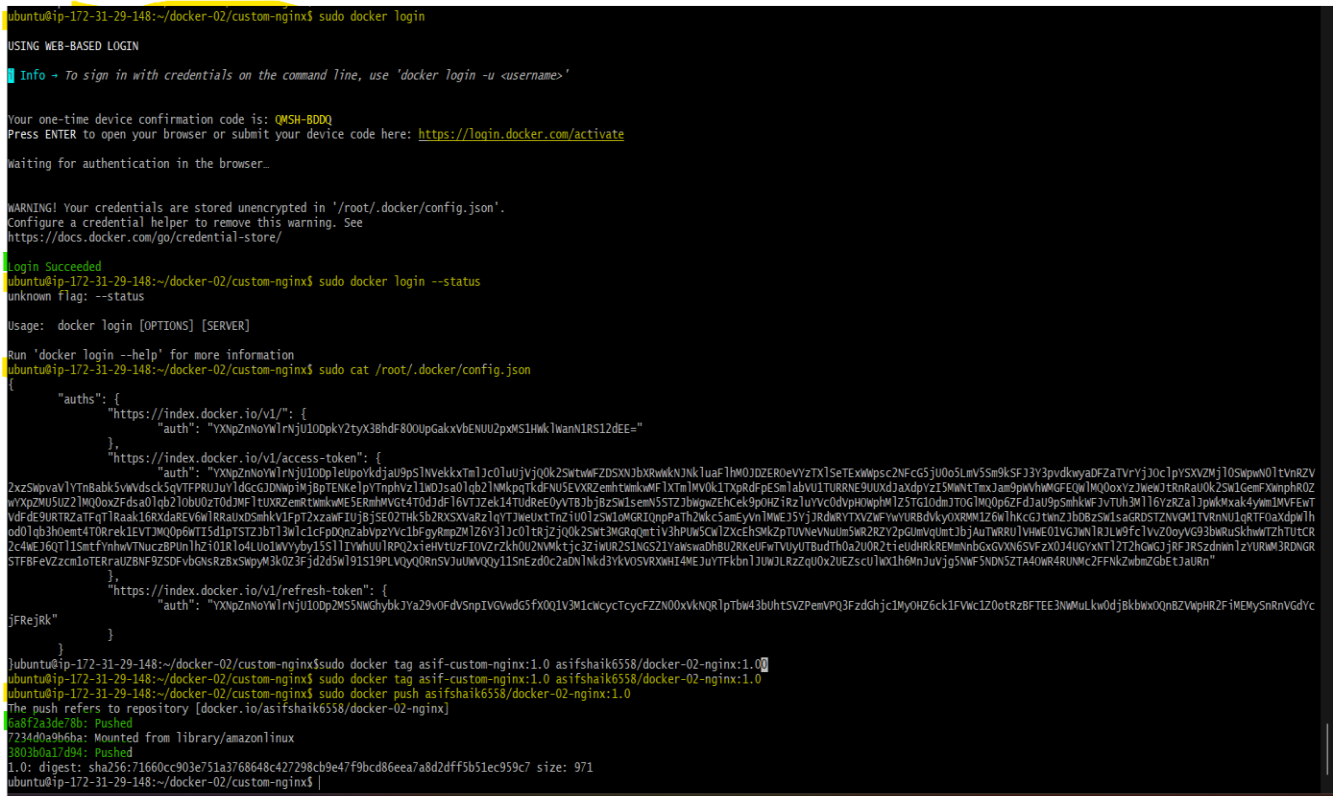
```
sudo docker push asifshaik6558/docker-02-nginx:1.0
```

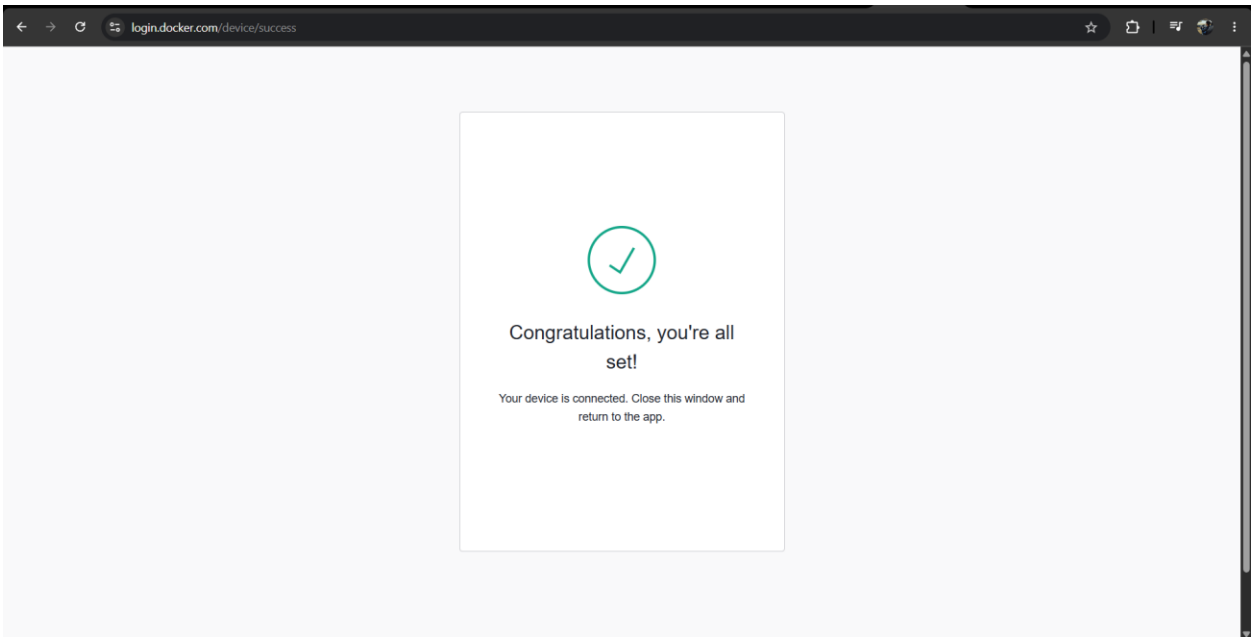
The image layers were uploaded successfully. Docker reused the Amazon Linux layer where possible.

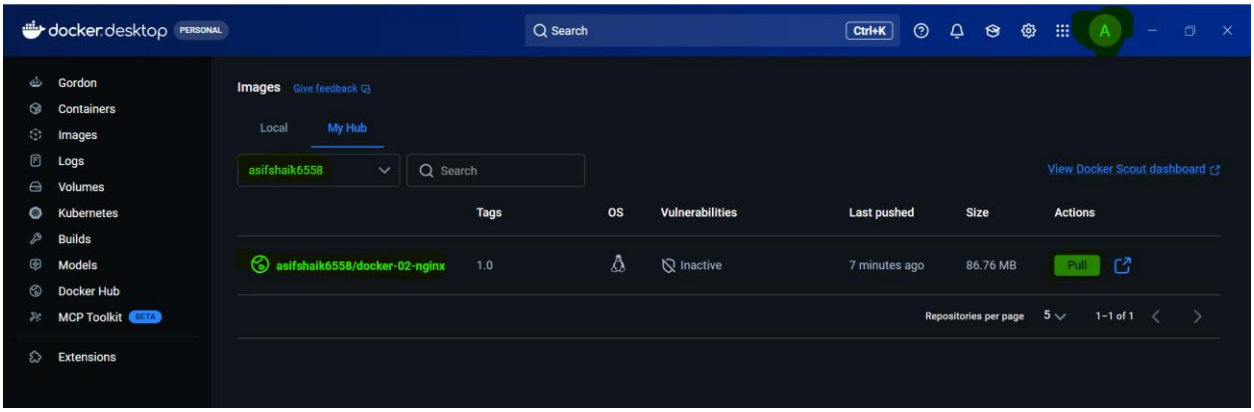
Step 4: Verify the repository

Opened the Docker Hub repository to check the uploaded image and tag.

Repository: https://hub.docker.com/r/asifshaik6558/docker-02-nginx







Observation

- The image was tagged with the Docker Hub username and repository name.
- Docker uploaded the required image layers successfully.
- Tag `1.0` was pushed with the reported image digest.

Conclusion

Successfully pushed the custom Nginx Docker image to Docker Hub. The image is available in the `asifshaik6558/docker-02-nginx` repository with tag `1.0`.

Task 6: Push Custom Nginx Image to AWS ECR.

1. Objective

To create an Amazon ECR private repository and push the custom Nginx Docker image, built using Amazon Linux, from an Ubuntu EC2 instance to AWS ECR.

2. Tools and Resources Used

Resource	Actual value used
EC2 operating system	Ubuntu
EC2 working directory	`~/docker-02/custom-nginx`
AWS Region	`us-east-1` (North Virginia)
AWS CLI	Version 2.31.35
Docker image created	`asif-custom-nginx:1.0`
Docker Hub image	`asifshaik6558/docker-02-nginx:1.0`
IAM role	`EC2-ECR-Push-Role`
IAM policies attached	`AmazonEC2ContainerRegistryPowerUser`, then `AmazonEC2ContainerRegistryFullAccess`
ECR repository	`docker-02-nginx`
ECR image tag	`1.0`

3. Steps Used

Step 1: Install and verify AWS CLI

Checked the AWS CLI version on the Ubuntu EC2 instance.

```
aws --version
```

Observation: AWS CLI was installed and returned version `2.31.35`.

Step 2: Create an IAM role and attach it to EC2

Using the AWS Management Console:

1. Created the IAM role `EC2-ECR-Push-Role`.
2. Attached the role to the EC2 instance.
3. Initially attached `AmazonEC2ContainerRegistryPowerUser`.
4. When repository creation was denied, added `AmazonEC2ContainerRegistryFullAccess`.

Verified the role credentials:

```
aws sts get-caller-identity
```

Observation: AWS returned account ID `331116047691` and the assumed role `EC2-ECR-Push-Role`.

Step 3: Configure the AWS region

```
aws configure set region us-east-1
```

Observation: The default AWS CLI region was configured as `us-east-1`, North Virginia.

Step 4: Create an ECR repository

```
aws ecr create-repository \
  --repository-name docker-02-nginx \
  --region us-east-1
```

Observation: The private ECR repository was created successfully.

Repository URI:

```
331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx
```

Step 5: Log in to AWS ECR

```
aws ecr get-login-password --region us-east-1 | sudo docker login --username AWS --password-stdin 331116047691.dkr.ecr.us-east-1.amazonaws.com
```

Observation: Docker login succeeded. AWS CLI generated the temporary authentication password, which was passed to Docker without displaying it.

Step 6: Tag the custom Docker image

```
sudo docker tag asif-custom-nginx:1.0 331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx:1.0
```

Observation: The existing local image was tagged with the ECR repository URI and version `1.0`. Tagging did not rebuild the image.

Step 7: Verify the image tag locally

```
sudo docker images
```

Observation: The image appeared with the ECR repository URI and tag `1.0`. Its image ID was `71660cc903e7`.

Step 8: Push the image to ECR

```
sudo docker push 331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx:1.0
```

Actual result:

```
The push refers to repository [331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx]
6a8f2a3de78b: Pushed
7234d0a9b6ba: Pushed
3803b0a17d94: Pushed
1.0: digest: sha256:71660cc903e751a3768648c427298cb9e47f9bcd86eea7a8d2dff5b51ec959c7 size: 971
```

Observation: All three image layers were pushed successfully, and ECR returned the image digest.

Step 9: Verify the repository image (optional)

```
aws ecr describe-images \
  --repository-name docker-02-nginx \
  --region us-east-1
```

This command checks the image details stored in ECR.

```
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ sudo apt install awscli
Installing:
  awscli

Installing dependencies:
  docutils-common  libharfbuzz0b   libjpeg-turbo8  liblerc4       libpaper2     libtiff6       libwebpmux3      python3-docutils  python3-prompt-toolkit  python3-ruamel.yaml.clib
  libdeflate0      libimagequant0  libjpeg8        libopenjp2-7   libraqm0      libwebp7       python3-awscrt   python3-olefile   python3-roman-numerals  python3-wcwidth
  libgraphite2-3   libjbig0        liblcms2-2      libpaper-utils  libsharpyuv0  libwebpdemux2  python3-colorama  python3-pil       python3-ruamel.yaml

Suggested packages:
  liblcms2-utils  docutils-doc  fonts-linuxlibertine  | ttf-linux-libertine  texlive-lang-french  texlive-latex-base  texlive-latex-recommended  python-pil-doc

Summary:
  Upgrading: 0, Installing: 30, Removing: 0, Not Upgrading: 49
  Download size: 16.0 MB
  Space needed: 148 MB / 11.6 GB available

Continue? [Y/n] y
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 python3-awscrt amd64 0.28.4+dfsg-1 [954 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 python3-colorama all 0.4.6-4build1 [32.2 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 docutils-common all 0.22.4+dfsg-1 [130 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 python3-roman-numerals all 4.1.0-1 [8660 B]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 python3-docutils all 0.22.4+dfsg-1 [439 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 python3-wcwidth all 0.2.14+dfsg1-1build1 [26.5 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 python3-prompt-toolkit all 3.0.52-2 [258 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 python3-ruamel.yaml.clib amd64 0.2.15+ds-1build1 [140 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 python3-ruamel.yaml all 0.18.10+ds-1build1 [127 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 awscli all 2.31.35-1 [11.1 MB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libdeflate0 amd64 1.23-2ubuntu1 [48.9 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 libgraphite2-3 amd64 1.3.14-11ubuntu1.1 [72.5 kB]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libharfbuzz0b amd64 12.3.2-2 [525 kB]
Get:14 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libimagequant0 amd64 4.4.1-1 [264 kB]
Get:15 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libjpeg-turbo8 amd64 2.1.5-4ubuntu4 [162 kB]
Get:16 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libjpeg8 amd64 8c-2ubuntu12 [2154 B]
Get:17 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 liblcms2-2 amd64 2.17-1ubuntu0.2 [172 kB]
Get:18 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 liblerc4 amd64 4.0.0+ds-5ubuntu2 [212 kB]
Get:19 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libpaper2 amd64 2.2.5-0.3maysync1 [17.3 kB]
Get:20 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libpaper-utils amd64 2.2.5-0.3maysync1 [15.6 kB]
Get:21 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libraqm0 amd64 0.10.4-1 [15.6 kB]
Get:22 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libsharpyuv0 amd64 1.5.0-0.1build1 [17.6 kB]
Get:23 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libjbig0 amd64 2.1-6.1ubuntu3 [30.4 kB]
Get:24 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libwebp7 amd64 1.5.0-0.1build1 [268 kB]
Get:25 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 libtiff6 amd64 4.7.0-3ubuntu5 [209 kB]
Get:26 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libwebpdemux2 amd64 1.5.0-0.1build1 [12.9 kB]
Get:27 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libwebpmux3 amd64 1.5.0-0.1build1 [26.4 kB]
Get:28 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 python3-olefile all 0.47-1build1 [37.2 kB]
Get:29 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 libopenjp2-7 amd64 2.5.4-1ubuntu0.1 [190 kB]
Get:30 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 python3-pil amd64 12.1.1-2ubuntu1.3 [489 kB]
Fetched 16.0 MB in 0s (105 MB/s)
Preconfiguring packages ...
Selecting previously unselected package python3-awscrt.
(Reading database ... 124517 files and directories currently installed.)
Preparing to unpack .../00-python3-awscrt_0.28.4+dfsg-1_amd64v3.deb ...
Unpacking python3-awscrt (0.28.4+dfsg-1) ...
Selecting previously unselected package python3-colorama.
```

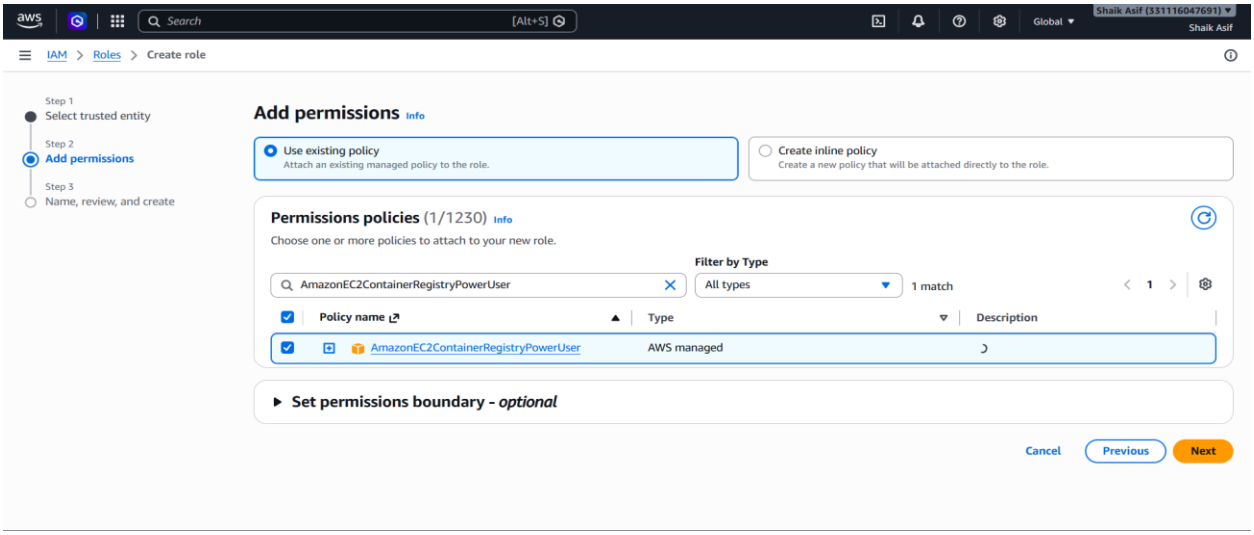
```
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ aws sts get-caller-identity
{
    "UserId": "AROAU2GAN5VF5OCGLGOCU:i-0c28387df6bacafb1",
    "Account": "331116047691",
    "Arn": "arn:aws:sts::331116047691:assumed-role/EC2-ECR-Push-Role/i-0c28387df6bacafb1"
}
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ aws ec2 describe-availability-zones \
  --query 'AvailabilityZones[0].RegionName' \
  --output text
```

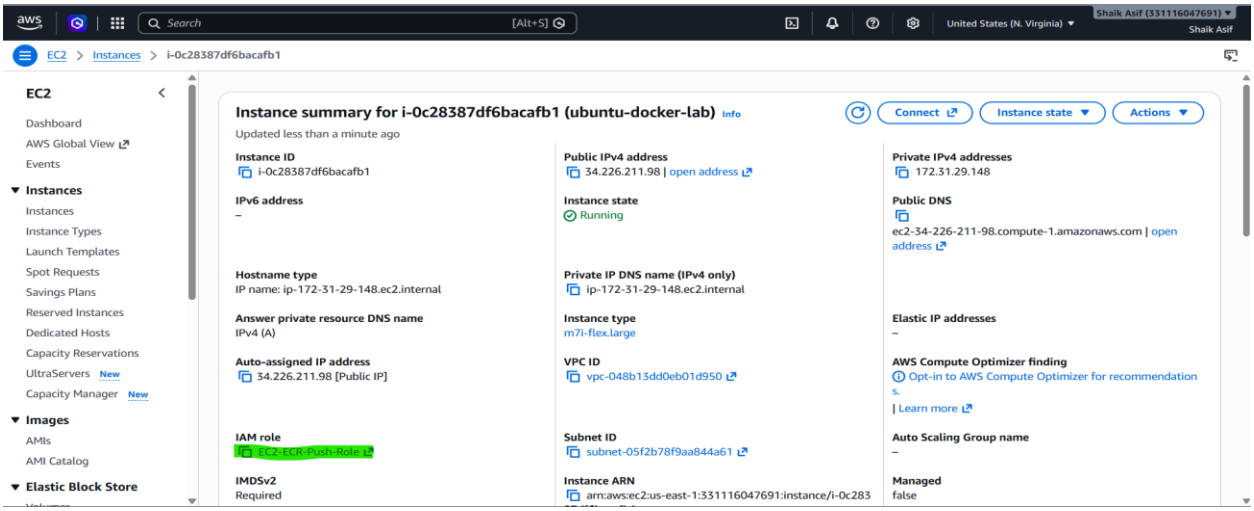
```
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ aws ecr create-repository \
  --repository-name docker-02-nginx \
  --region us-east-1
{
    "repository": {
        "repositoryArn": "arn:aws:ecr:us-east-1:331116047691:repository/docker-02-nginx",
        "registryId": "331116047691",
        "repositoryName": "docker-02-nginx",
        "repositoryUri": "331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx",
        "createdAt": "2026-10-13T12:21:20.895000+00:00",
        "imageTagMutability": "MUTABLE",
        "imageScanningConfiguration": {
            "scanOnPush": false
        },
        "encryptionConfiguration": {
            "encryptionType": "AES256"
        }
    }
}
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ aws ecr get-login-password --region us-east-1 | sudo docker login --username AWS --password-stdin 331116047691.dkr.ecr.us-east-1.amazonaws.com

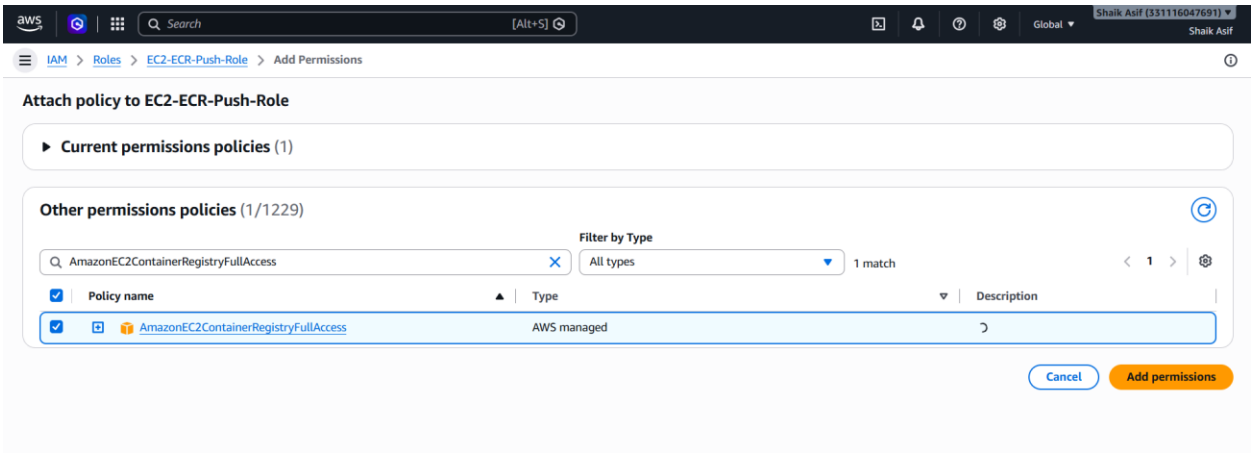
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

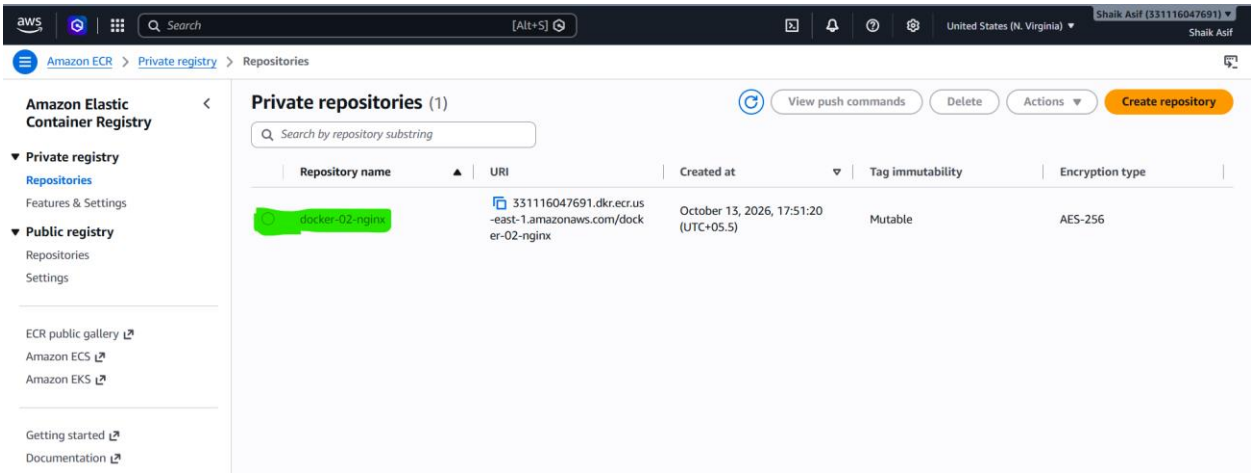
Login Succeeded
```

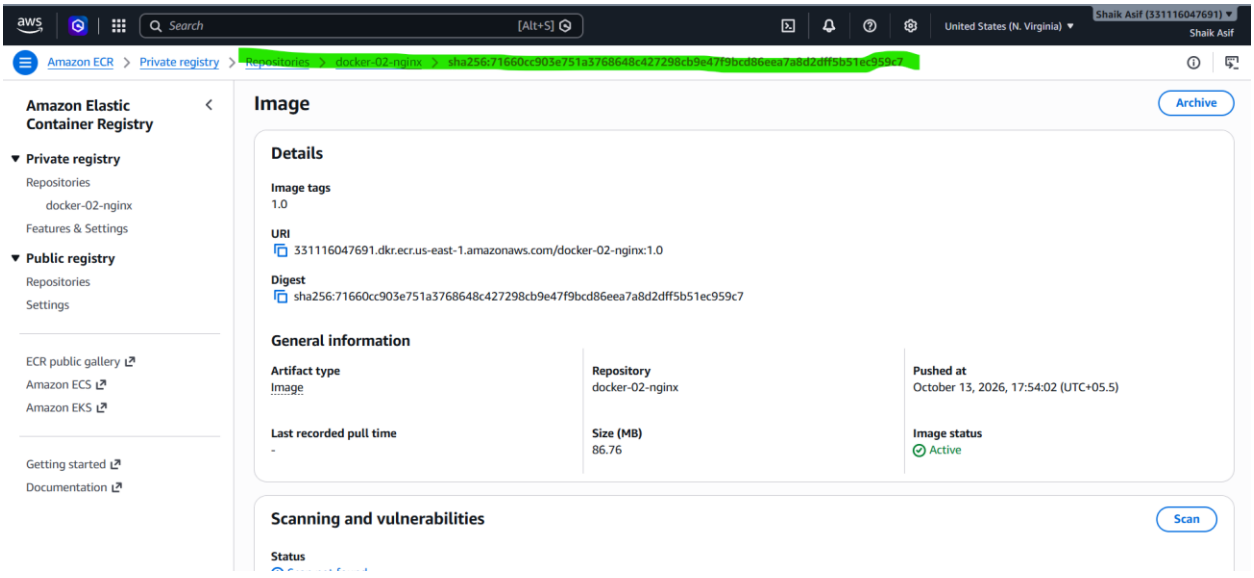
```
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ sudo docker tag asif-custom-nginx:1.0 331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx:1.0
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ sudo docker images
IMAGE                                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx:1.0   71660cc903e7       354MB         86.8MB    U
alpine:latest                                                  294b883cb724        13MB         3.94MB
amazonlinux:latest                                             8ed3c0a99684       227MB         59.3MB
asif-custom-nginx:1.0                                          71660cc903e7       354MB         86.8MB    U
asifshaik6558/docker-02-nginx:1.0                              71660cc903e7       354MB         86.8MB    U
hello-world:latest                                             5e2309035332      25.9kB         9.49kB
jenkins/jenkins:lts-jdk21                                      a660310e39ad       720MB          246MB
nginx:latest                                                   f9ea18bfa4fa       243MB         66.7MB    U
sonarqube:community                                            c0f1160bccfa      2.55GB          1.1GB
tomcat:10.1                                                    574ed18cff4d       640MB          174MB    U
ubuntu:24.04                                                   534baea6a22c       119MB         31.7MB
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ sudo docker push 331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx:1.0
The push refers to repository [331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx]
6a8f2a3de78b: Pushed
7234d0a9b6ba: Pushed
3803b0a17d94: Pushed
1.0: digest: sha256:71660cc903e751a3768648c427298cb9e47f9bcd86eea7a8d2dff5b51ec959c7 size: 971
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$
ubuntu@ip-172-31-29-148:~/docker-02/custom-nginx$ aws ecr describe-images \
  --repository-name docker-02-nginx \
  --region us-east-1
{
    "imageDetails": [
        {
            "registryId": "331116047691",
            "repositoryName": "docker-02-nginx",
            "imageDigest": "sha256:71660cc903e751a3768648c427298cb9e47f9bcd86eea7a8d2dff5b51ec959c7",
            "imageTags": [
                "1.0"
            ],
            "imageSizeInBytes": 86762245,
            "imagePushedAt": "2026-10-13T12:24:02.289000+00:00",
            "imageManifestMediaType": "application/vnd.oci.image.manifest.v1+json",
            "artifactMediaType": "application/vnd.oci.image.config.v1+json"
        }
    ]
}
```











4. Observation

- AWS CLI accessed AWS using the EC2 IAM role.

- The private ECR repository `docker-02-nginx` was created in `us-east-1`.
- Docker authenticated successfully with AWS ECR.
- The custom Nginx image was tagged as `1.0`.
- The Docker push completed successfully with a returned image digest.

5. Conclusion

Successfully pushed the custom Nginx image built from `amazonlinux:latest` to the private Amazon ECR repository `docker-02-nginx` in the North Virginia region. The image is available at:

`331116047691.dkr.ecr.us-east-1.amazonaws.com/docker-02-nginx:1.0`